

Decreasing Patterns using Nested Loops

Submitted By

Name	Rudra Pratap Singh
SAP ID	590032811
Batch	16

Problem Statement

Write a C program that displays a menu and uses a switch statement to print the "Decreasing pattern" selected by the user. Nested loops must be used and arrays must not be used. Each pattern is implemented in a separate parameterless function with no return value.

The menu offered by the program is:

- 1. Decreasing right-angled triangle
- 2. Decreasing natural-number pattern
- 3. Decreasing repeated-number pattern
- 4. Reverse-number triangle

Concepts Used

- Menu-driven programming with a do-while loop, which keeps showing the menu until the user selects Exit.
- The switch statement to call the function that matches the user's choice.
- Nested for loops: the outer loop controls the rows and the inner loop controls the elements printed in each row.
- User-defined functions with no parameters and no return value (void function(void)).
- Input validation: the number of rows must be a positive integer, otherwise an error message is displayed.

Main Program (Menu) Algorithm

1. Start.
2. Declare an integer variable choice.
3. Display the menu with options 1 to 5.
4. Read the value of choice.
5. Use a switch statement on choice: case 1 calls rightTriangle(); case 2 calls naturalNumberPattern(); case 3 calls repeatedNumberPattern(); case 4 calls reverseNumberTriangle(); case 5 displays the exit message; default displays an invalid choice message.
6. If choice is not equal to 5, go back to step 3.
7. Stop.

Tools Used

Language	C (C99 / C11)
Compiler	GCC 13.3.0 (gcc -Wall -Wextra)
Source file	decreasing_patterns.c
Compile command	gcc decreasing_patterns.c -o decreasing_patterns
Run command	./decreasing_patterns

Activity 1: Decreasing Right-Angled Triangle

Aim

To read the number of rows n and print a decreasing right-angled triangle made of asterisks.

Logic

Row 1 contains n asterisks, row 2 contains $n - 1$, and so on until the last row contains 1. The outer loop counter i runs from n down to 1 and the inner loop prints i asterisks.

Algorithm (function rightTriangle())

1. Start the function rightTriangle().
2. Declare integer variables n , i and j .
3. Read the number of rows n from the user.
4. If $n \leq 0$, display an error message and return from the function.
5. Set $i = n$.
6. Repeat steps 7 to 10 while $i \geq 1$.
7. Set $j = 1$.
8. Repeat while $j \leq i$: print "*" and increase j by 1.
9. Print a newline character to move to the next row.
10. Decrease i by 1 and go back to step 6.
11. End the function.

Test-Case Table

Test ID	Input (n)	Expected Output	Actual Output	Result
A1-T1	4	* * * * * * * * * *	* * * * * * * * * *	PASS
A1-T2	1	*	*	PASS
A1-T3	6	* *	* *	PASS
A1-T4	0	Error: Number of rows must be a positive integer.	Error: Number of rows must be a positive integer.	PASS
A1-T5	-3	Error: Number of rows must be a positive integer.	Error: Number of rows must be a positive integer.	PASS

Program Output (Menu Choice 1)

```
Enter your choice (1-5): 1
Enter the number of rows (n): 4
* * * *
* * *
* *
*
```

Activity 2: Decreasing Natural-Number Pattern

Aim

To read the number of rows n and print natural numbers starting from 1 in each row, reducing the count of numbers by one in every following row.

Logic

The outer loop counter i (from n down to 1) decides how many numbers appear in the current row. The inner loop counter j runs from 1 up to i and prints j .

Algorithm (function naturalNumberPattern())

1. Start the function naturalNumberPattern().
2. Declare integer variables n , i and j .
3. Read the number of rows n from the user.
4. If $n \leq 0$, display an error message and return from the function.
5. Set $i = n$.
6. Repeat steps 7 to 10 while $i \geq 1$.
7. Set $j = 1$.
8. Repeat while $j \leq i$: print the value of j followed by a space and increase j by 1.
9. Print a newline character to move to the next row.
10. Decrease i by 1 and go back to step 6.
11. End the function.

Test-Case Table

Test ID	Input (n)	Expected Output	Actual Output	Result
A2-T1	4	1 2 3 4 1 2 3 1 2 1	1 2 3 4 1 2 3 1 2 1	PASS
A2-T2	1	1	1	PASS
A2-T3	6	1 2 3 4 5 6 1 2 3 4 5 1 2 3 4 1 2 3 1 2 1	1 2 3 4 5 6 1 2 3 4 5 1 2 3 4 1 2 3 1 2 1	PASS
A2-T4	0	Error: Number of rows must be a positive integer.	Error: Number of rows must be a positive integer.	PASS
A2-T5	-3	Error: Number of rows must be a positive integer.	Error: Number of rows must be a positive integer.	PASS

Program Output (Menu Choice 2)

```
Enter your choice (1-5): 2
Enter the number of rows (n): 4
1 2 3 4
1 2 3
1 2
1
```

Activity 3: Decreasing Repeated-Number Pattern

Aim

To read the number of rows n and print the value n in the first row, decreasing the value by one in every next row, where each value is printed as many times as its own value.

Logic

The outer loop counter i (from n down to 1) is both the value to be printed and the number of times it is printed. The inner loop simply prints i , exactly i times.

Algorithm (function repeatedNumberPattern())

1. Start the function repeatedNumberPattern().
2. Declare integer variables n , i and j .
3. Read the number of rows n from the user.
4. If $n \leq 0$, display an error message and return from the function.
5. Set $i = n$.
6. Repeat steps 7 to 10 while $i \geq 1$.
7. Set $j = 1$.
8. Repeat while $j \leq i$: print the value of i followed by a space and increase j by 1.
9. Print a newline character to move to the next row.
10. Decrease i by 1 and go back to step 6.
11. End the function.

Test-Case Table

Test ID	Input (n)	Expected Output	Actual Output	Result
A3-T1	4	4 4 4 4 3 3 3 2 2 1	4 4 4 4 3 3 3 2 2 1	PASS
A3-T2	1	1	1	PASS
A3-T3	6	6 6 6 6 6 6 5 5 5 5 5 4 4 4 4 3 3 3 2 2 1	6 6 6 6 6 6 5 5 5 5 5 4 4 4 4 3 3 3 2 2 1	PASS
A3-T4	0	Error: Number of rows must be a positive integer.	Error: Number of rows must be a positive integer.	PASS
A3-T5	-3	Error: Number of rows must be a positive integer.	Error: Number of rows must be a positive integer.	PASS

Program Output (Menu Choice 3)

```
Enter your choice (1-5): 3
Enter the number of rows (n): 4
4 4 4 4
3 3 3
2 2
1
```

Activity 4: Reverse-Number Triangle

Aim

To read the number of rows n and print, in each row, natural numbers in descending order starting from the number of elements in that row and ending at 1.

Logic

The outer loop counter i (from n down to 1) gives the starting number of the row. The inner loop counter j starts at i and counts down to 1, printing j each time.

Algorithm (function reverseNumberTriangle())

1. Start the function reverseNumberTriangle().
2. Declare integer variables n , i and j .
3. Read the number of rows n from the user.
4. If $n \leq 0$, display an error message and return from the function.
5. Set $i = n$.
6. Repeat steps 7 to 10 while $i \geq 1$.
7. Set $j = i$.
8. Repeat while $j \geq 1$: print the value of j followed by a space and decrease j by 1.
9. Print a newline character to move to the next row.
10. Decrease i by 1 and go back to step 6.
11. End the function.

Test-Case Table

Test ID	Input (n)	Expected Output	Actual Output	Result
A4-T1	4	4 3 2 1 3 2 1 2 1 1	4 3 2 1 3 2 1 2 1 1	PASS
A4-T2	1	1	1	PASS
A4-T3	6	6 5 4 3 2 1 5 4 3 2 1 4 3 2 1 3 2 1 2 1 1	6 5 4 3 2 1 5 4 3 2 1 4 3 2 1 3 2 1 2 1 1	PASS
A4-T4	0	Error: Number of rows must be a positive integer.	Error: Number of rows must be a positive integer.	PASS
A4-T5	-3	Error: Number of rows must be a positive integer.	Error: Number of rows must be a positive integer.	PASS

Program Output (Menu Choice 4)

```
Enter your choice (1-5): 4
Enter the number of rows (n): 4
4 3 2 1
3 2 1
2 1
1
```

Source Code

File name: decreasing_patterns.c

```
/*
 * Program : Decreasing Patterns (HDOC Day-25)
 * Name    : Rudra Pratap Singh
 * SAP ID  : 590032811
 * Batch   : 16
 *
 * Description:
 * Displays a menu and, using a switch statement, prints the
 * decreasing pattern chosen by the user. Every pattern is
 * implemented in its own parameterless function with no return
 * value. Only nested loops are used (no arrays).
 */

#include <stdio.h>

/* Function prototypes */
void rightTriangle(void);
void naturalNumberPattern(void);
void repeatedNumberPattern(void);
void reverseNumberTriangle(void);

int main(void)
{
    int choice;

    do
    {
        printf("\n===== DECREASING PATTERNS MENU =====\n");
        printf("1. Decreasing right-angled triangle\n");
        printf("2. Decreasing natural-number pattern\n");
        printf("3. Decreasing repeated-number pattern\n");
        printf("4. Reverse-number triangle\n");
        printf("5. Exit\n");
        printf("=====\n");
        printf("Enter your choice (1-5): ");

        if (scanf("%d", &choice) != 1)
        {
            printf("Invalid input! Please enter a number.\n");
            return 1;
        }

        switch (choice)
        {
            case 1:
                rightTriangle();
                break;
            case 2:
                naturalNumberPattern();
                break;
            case 3:
                repeatedNumberPattern();
                break;
            case 4:
                reverseNumberTriangle();
                break;
            case 5:
                printf("Exiting the program. Thank you!\n");
                break;
            default:
                printf("Invalid choice! Please select between 1 and 5.\n");
        }
    } while (choice != 5);
}
```

```
        return 0;
    }

    /* Pattern 1: Decreasing right-angled triangle of asterisks */
    void rightTriangle(void)
    {
        int n, i, j;

        printf("Enter the number of rows (n): ");
        scanf("%d", &n);

        if (n <= 0)
        {
            printf("Number of rows must be a positive integer.\n");
            return;
        }

        for (i = n; i >= 1; i--)          /* i = stars in current row */
        {
            for (j = 1; j <= i; j++)
            {
                printf("* ");
            }
            printf("\n");
        }
    }

    /* Pattern 2: Decreasing natural-number pattern (1 2 3 ... k) */
    void naturalNumberPattern(void)
    {
        int n, i, j;

        printf("Enter the number of rows (n): ");
        scanf("%d", &n);

        if (n <= 0)
        {
            printf("Number of rows must be a positive integer.\n");
            return;
        }

        for (i = n; i >= 1; i--)          /* i = numbers in current row */
        {
            for (j = 1; j <= i; j++)
            {
                printf("%d ", j);
            }
            printf("\n");
        }
    }

    /* Pattern 3: Decreasing repeated-number pattern (k printed k times) */
    void repeatedNumberPattern(void)
    {
        int n, i, j;

        printf("Enter the number of rows (n): ");
        scanf("%d", &n);

        if (n <= 0)
        {
            printf("Number of rows must be a positive integer.\n");
            return;
        }
    }
```

```
    for (i = n; i >= 1; i--)          /* i = value to print, and its count */
    {
        for (j = 1; j <= i; j++)
        {
            printf("%d ", i);
        }
        printf("\n");
    }
}

/* Pattern 4: Reverse-number triangle (k k-1 ... 1) */
void reverseNumberTriangle(void)
{
    int n, i, j;

    printf("Enter the number of rows (n): ");
    scanf("%d", &n);

    if (n <= 0)
    {
        printf("Number of rows must be a positive integer.\n");
        return;
    }

    for (i = n; i >= 1; i--)          /* i = starting value of current row */
    {
        for (j = i; j >= 1; j--)
        {
            printf("%d ", j);
        }
        printf("\n");
    }
}
```

Compilation and Execution

The program was compiled with all warnings enabled. It compiled with no errors and no warnings.

```
$ gcc -Wall -Wextra decreasing_patterns.c -o decreasing_patterns
$ ./decreasing_patterns
```

Sample Run 1: All four patterns with n = 4

```
===== DECREASING PATTERNS MENU =====
1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
5. Exit
=====
Enter your choice (1-5): 1
Enter the number of rows (n): 4
* * * *
* * *
* *
*
*

===== DECREASING PATTERNS MENU =====
1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
5. Exit
=====
Enter your choice (1-5): 2
Enter the number of rows (n): 4
1 2 3 4
1 2 3
1 2
1

===== DECREASING PATTERNS MENU =====
1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
5. Exit
=====
Enter your choice (1-5): 3
Enter the number of rows (n): 4
4 4 4 4
3 3 3
2 2
1

===== DECREASING PATTERNS MENU =====
1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
5. Exit
=====
Enter your choice (1-5): 4
Enter the number of rows (n): 4
4 3 2 1
3 2 1
2 1
1
```

```
===== DECREASING PATTERNS MENU =====
1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
5. Exit
=====
Enter your choice (1-5): 5
Exiting the program. Thank you!
```

Sample Run 2: Invalid menu choice and invalid number of rows

```
===== DECREASING PATTERNS MENU =====
1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
5. Exit
=====
Enter your choice (1-5): 9
Invalid choice! Please select between 1 and 5.

===== DECREASING PATTERNS MENU =====
1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
5. Exit
=====
Enter your choice (1-5): 1
Enter the number of rows (n): 0
Number of rows must be a positive integer.

===== DECREASING PATTERNS MENU =====
1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
5. Exit
=====
Enter your choice (1-5): 5
Exiting the program. Thank you!
```

Menu Test Cases

Test ID	Menu Choice	Expected Output	Actual Output	Result
M-T1	9	Invalid choice! Please select between 1 and 5.	Invalid choice! Please select between 1 and 5.	PASS
M-T2	5	Exiting the program. Thank you!	Exiting the program. Thank you!	PASS

Test Summary

Module	Test Cases	Passed	Status
Activity 1 - Decreasing Right-Angled Triangle	5	5	All Passed
Activity 2 - Decreasing Natural-Number Pattern	5	5	All Passed
Activity 3 - Decreasing Repeated-Number Pattern	5	5	All Passed
Activity 4 - Reverse-Number Triangle	5	5	All Passed
Menu handling (switch / default)	2	2	All Passed
Total	22	22	100%

Conclusion

A menu-driven C program was written that prints four decreasing patterns using nested loops, a switch statement and parameterless void functions, without using any arrays. An algorithm and a test-case table were prepared for each activity, the program was compiled with GCC, and it was executed against every test case. The actual output matched the expected output in all 22 test cases, including the boundary case $n = 1$ and the invalid inputs $n = 0$ and $n = -3$.